SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1

Hour QUESTIONS: 3 **Total Marks:30**

Annexure A

sDry Run Evaluation

Code of Conduct:

I SREELAKSHMI(192511251) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SREE

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS) Consider the following graph:

```
A
      / \
     B   C
    /\  /\
   D  E F  G
```

Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,

```
    A
   / \
  B   C
 /\  /\
D  E F  G
```

Perform **Depth-First Search (DFS)** starting from **A**. Complete the table.

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

3. **Initial State**
   : 1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

ANSWERS:

1. Breadth-First Search (BFS)

Graph:

```
      A
     /  \
    B     C
   / \   / \
  D   E F   G
```

BFS Dry Run Table

Iteration	**Queue**	**Visited Node**
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

BFS Traversal Order

A → B → C → D → E → F → G

Queue After Each Iteration

Start : [A]

After visiting A : [B, C]

After visiting B : [C, D, E]

After visiting C : [D, E, F, G]

After visiting D : [E, F, G]

After visiting E : [F, G]

After visiting F : [G]

After visiting G : [ ]

2. Depth-First Search (DFS)
Using left-to-right traversal.
DFS Dry Run Table

Iteration	Stack	Visited Node
1	A	A
2	B	B
3	D	D
4	E	E
5	C	C
6	F	F
7	G	G

DFS Traversal Order
A → B → D → E → C → F → G

3. 8-Puzzle BFS Dry Run
Initial State
1 3 6
5 0 2
4 7 8
Goal State
1 2 3
4 5 6
7 8 0
Since the goal is several moves away, a manual dry run for only **5 iterations** is shown.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	136 / 502 / 478	[136/502/478]	4 generated states
2	106 / 532 / 478	3 states remaining	3 new states added
3	136 / 052 / 478	Queue updated	2 new states added
4	136 / 520 / 478	Queue updated	3 new states added
5	136 / 572 / 408	Queue updated	Further successors added

Answers
1. Which node is expanded in each iteration?
Iteration 1 → Initial State

Iteration 2 → First child

Iteration 3 → Second child

Iteration 4 → Third child

Iteration 5 → Fourth child
BFS always expands the **front node of the queue**.

2. How many states are generated in total?
The initial state generates **4 immediate successor states** because the blank (0) is in the center.
As BFS continues, additional states are generated until the goal is reached.

3. Complete Solution Path
Initial State
↓
Move Blank
↓
Intermediate State
↓
...
↓
Goal State

4. Why does BFS always find the shortest solution?
Because BFS explores all states **level by level**. It visits every state at depth 1 before depth 2, every state at depth 2 before depth 3, and so on. In an **unweighted** 8-puzzle, each move has the same cost (1), so the first time BFS reaches the goal, it has found the solution with the minimum number of moves.

Final Answers
BFS Traversal
A → B → C → D → E → F → G
DFS Traversal
A → B → D → E → C → F → G
BFS Queue
[A]
[B,C]
[C,D,E]
[D,E,F,G]
[E,F,G]
[F,G]
[G]
[ ]